



JECRCTM
UNIVERSITY
BUILD YOUR WORLD

Prompt Engineering for C and C++ 2026-2027

Course: B.Tech CSE (Core)

Registration number: 25BCON1594

Section: I

Semester: III

Submitted to: Mr. Ajay Sharma

Submitted by: Chehak Gupta

Session 08 Classwork: Prove Fixes with Specification-Driven Tests

1. Activity A & B: The Red Stage (Test Execution)

Requirement: Write 9 tests from the required behavior (not implementation), commit tests and failing output before any fix, and explain each result.

RED STAGE Terminal Output Showing 4 Failures

Original Code:



```
student_utils.py  test_student_utils.py X
CW > test_student_utils.py > ...
1  import unittest
2  from student_utils import calculate_average, is_passing, get_grade
3
4  class TestStudentUtils(unittest.TestCase):
5      def test_ordinary_arithmetic(self):
6          self.assertEqual(calculate_average([80, 90, 100]), 90.0)
7
8      def test_single_element_no_crash(self):
9          self.assertEqual(calculate_average([40]), 40.0)
10
11     def test_clearly_above_mark(self):
12         self.assertTrue(is_passing(41))
13
14     def test_exactly_at_pass_mark(self):
15         self.assertTrue(is_passing(40))
16
17     def test_just_below_mark(self):
18         self.assertFalse(is_passing(39))
19
20     def test_top_band(self):
21         self.assertEqual(get_grade(95), "Distinction")
22
23     def test_first_band(self):
24         self.assertEqual(get_grade(60), "First")
25
26     def test_second_band(self):
27         self.assertEqual(get_grade(45), "Second")
28
29     def test_below_all_bands(self):
30         self.assertEqual(get_grade(30), "Fail")
31
32     def test_perfect_score_upper_bounds(self):
33         self.assertEqual(get_grade(100), "Distinction")
34
35     if __name__ == '__main__':
36         unittest.main(argv=[""], exit=False)
```

Output:

```
Problems Output Debug Console Terminal Ports
PS C:\Users\DESKTOP\OneDrive\Documents\sessin08 & C:\Users\DESKTOP\AppData\Local\Programs\Python\Python314\python.exe c:\Users\DESKTOP\OneDrive\Documents\sessin08\CW\test_student_utils.py
..F..FF.EF
=====
ERROR: test_single_element_no_crash (__main__.TestStudentUtils.test_single_element_no_crash)
-----
Traceback (most recent call last):
  File "c:\Users\DESKTOP\OneDrive\Documents\sessin08\CW\test_student_utils.py", line 9, in test_single_element_no_crash
    self.assertEqual(calculate_average(140), 40.0)
    ~~~~~^~~~~~
File "c:\Users\DESKTOP\OneDrive\Documents\sessin08\CW\student_utils.py", line 7, in calculate_average
    return total / (len(marks) - 1)
           ^~~~~~
ZeroDivisionError: division by zero

=====
FAIL: test_exactly_at_pass_mark (__main__.TestStudentUtils.test_exactly_at_pass_mark)
-----
Traceback (most recent call last):
  File "c:\Users\DESKTOP\OneDrive\Documents\sessin08\CW\test_student_utils.py", line 15, in test_exactly_at_pass_mark
    self.assertTrue(is_passing(40))
                   ~~~~~^~~~~~
AssertionError: False is not true

=====
FAIL: test_ordinary_arithmetic (__main__.TestStudentUtils.test_ordinary_arithmetic)
-----
Traceback (most recent call last):
  File "c:\Users\DESKTOP\OneDrive\Documents\sessin08\CW\test_student_utils.py", line 6, in test_ordinary_arithmetic
    self.assertEqual(calculate_average([80, 90, 100]), 90.0)
    ~~~~~^~~~~~
AssertionError: 135.0 != 90.0

=====
FAIL: test_perfect_score_upper_bounds (__main__.TestStudentUtils.test_perfect_score_upper_bounds)
-----
Traceback (most recent call last):
  File "c:\Users\DESKTOP\OneDrive\Documents\sessin08\CW\test_student_utils.py", line 33, in test_perfect_score_upper_bounds
    self.assertEqual(get_grade(100), "Distinction")
    ~~~~~^~~~~~
AssertionError: 'First' != 'Distinction'
- First
+ Distinction

=====
FAIL: test_top_band (__main__.TestStudentUtils.test_top_band)
-----
Traceback (most recent call last):
  File "c:\Users\DESKTOP\OneDrive\Documents\sessin08\CW\test_student_utils.py", line 21, in test_top_band
    self.assertEqual(get_grade(95), "Distinction")
    ~~~~~^~~~~~
  File "c:\Users\DESKTOP\OneDrive\Documents\sessin08\CW\test_student_utils.py", line 21, in test_top_band
  File "c:\Users\DESKTOP\OneDrive\Documents\sessin08\CW\test_student_utils.py", line 21, in test_top_band
  File "c:\Users\DESKTOP\OneDrive\Documents\sessin08\CW\test_student_utils.py", line 21, in test_top_band
    self.assertEqual(get_grade(95), "Distinction")
    ~~~~~^~~~~~
  File "c:\Users\DESKTOP\OneDrive\Documents\sessin08\CW\test_student_utils.py", line 21, in test_top_band
    self.assertEqual(get_grade(95), "Distinction")
    ~~~~~^~~~~~
  File "c:\Users\DESKTOP\OneDrive\Documents\sessin08\CW\test_student_utils.py", line 21, in test_top_band
  File "c:\Users\DESKTOP\OneDrive\Documents\sessin08\CW\test_student_utils.py", line 21, in test_top_band
  File "c:\Users\DESKTOP\OneDrive\Documents\sessin08\CW\test_student_utils.py", line 21, in test_top_band
  File "c:\Users\DESKTOP\OneDrive\Documents\sessin08\CW\test_student_utils.py", line 21, in test_top_band
  File "c:\Users\DESKTOP\OneDrive\Documents\sessin08\CW\test_student_utils.py", line 21, in test_top_band
    self.assertEqual(get_grade(95), "Distinction")
    ~~~~~^~~~~~
  File "c:\Users\DESKTOP\OneDrive\Documents\sessin08\CW\test_student_utils.py", line 21, in test_top_band
    self.assertEqual(get_grade(95), "Distinction")
    ~~~~~^~~~~~
  File "c:\Users\DESKTOP\OneDrive\Documents\sessin08\CW\test_student_utils.py", line 21, in test_top_band
    self.assertEqual(get_grade(95), "Distinction")
    ~~~~~^~~~~~
  File "c:\Users\DESKTOP\OneDrive\Documents\sessin08\CW\test_student_utils.py", line 21, in test_top_band
    self.assertEqual(get_grade(95), "Distinction")
    ~~~~~^~~~~~
  File "c:\Users\DESKTOP\OneDrive\Documents\sessin08\CW\test_student_utils.py", line 21, in test_top_band
    self.assertEqual(get_grade(95), "Distinction")
    ~~~~~^~~~~~
AssertionError: 'First' != 'Distinction'
  File "c:\Users\DESKTOP\OneDrive\Documents\sessin08\CW\test_student_utils.py", line 21, in test_top_band
    self.assertEqual(get_grade(95), "Distinction")
```


2. Bug Mapping & AI Identification Prompt

Requirement: Use a chain prompt to identify and explain bugs without fixing them, then reproduce every claim.

AI Chain Prompt Used:

"Review my broken student_utils.py code. Identify only, then explain only. No fixes." AI

Response:

"I have identified three different bug classes in your module:

1. **Arithmetic:** Line 7 divides by `len(marks) - 1`, which gives the wrong average value and crashes for a single mark.
2. **Boundary:** Line 10 uses `if mark > PASS_MARK`, which rejects a mark of exactly 40.
3. **Unreachable Branch:** Line 19 checks `el if average >= 98` after checking `>= 68`. The highest band must be moved first, or all 90+ scores will be swallowed by the 60+ check."

Mapping Bugs to the Specification Tests:

Bug Class	The Fix Required	Specification Test that Exposed it
Arithmetic	Divide by <code>len(marks)</code>	<code>Calculate_average([80,90,100])</code> (Ordinary arithmetic) & <code>[40]</code>) (Single element)
Boundary	Use to evaluate exactly 40	<code>is_passing(40)</code> (Exactly at mark)
Unreachable Branch	Move the highest band (<code>>=90</code>) first	<code>get_grade(95)</code> (Top band)

3. Activity C: Fix and Go Green

Requirement: Fix the three bugs, run all tests to ensure they pass, and add one new test. Tests describe what the code should do, not what it currently does.

- Student-Designed Test Choice:
 - o **Chosen Input:** A student receives an average of exactly **100**.
 - o **Expected Behavior:** The program should return "Distinction" (Testing the upper extreme bounds of the highest band).

```
PS C:\Users\DESKTOP\OneDrive\Documents\session08> & C:/Users/DESKTOP/AppData/Local/Programs/Python/Python314/python.exe c:/Users/DESKTOP/OneDrive/Documents/session08/Q
*****
-----
Ran 10 tests in 0.000s
OK
PS C:\Users\DESKTOP\OneDrive\Documents\session08> 
```

4. Documentation & Commits

Requirement: Generate a Session 5 format commit message and audit the README. The failure evidence must exist before the implementation fix.

AI Prompt for Commit Message: "Generate a Session 5 format commit message summarizing fixes for arithmetic, boundary, and unreachable branch bugs."

My Actual Commit Message: "fix: Correct calculation, boundary logic, and unreachable grade branches in student_utils"

Audited README Testing Section:

Testing

- **Coverage:** Validates ordinary arithmetic, single-element arrays without crashing, strict boundary checks at pass marks, and complete grading bands.
- **Run Command:** `python test_student_utils.py`
- **Test Count:** 10 total tests (9 specification-driven, 1 student-designed).

5. Code Implementation

Fixed student_utils.py:

Python

```
PASS_MARK = 40
```

```
def calculate_average(marks):  
    total = 0  
    for m in marks:  
        total = total + m  
    return total / len(marks) # Fixed: Arithmetic bug
```

```
def is_passing(mark):  
    if mark >= PASS_MARK: # Fixed: Boundary bug  
        return True  
    return False
```

```
def get_grade(average):  
    if average >= 90: # Fixed: Unreachable branch moved up  
        return "Distinction"  
    elif average >= 60:  
        return "First"  
    elif average >= 45:  
        return "Second"  
    return "Fail"
```

test_student_utils.py:

Python

```
import unittest
```

```
from student_utils import calculate_average, is_passing, get_grade
```

```
class TestStudentUtils(unittest.TestCase):  
    # Tests matching the 9 Specification rows exactly[cite: 23, 31]  
    def test_ordinary_arithmetic(self):  
        self.assertEqual(calculate_average([80, 90, 100]), 90.0)  
  
    def test_single_element_no_crash(self):
```

```

        self.assertEqual(calculate_average([40]), 40.0)

    def test_clearly_above_mark(self):
        self.assertTrue(is_passing(41))

    def test_exactly_at_pass_mark(self):
        self.assertTrue(is_passing(40))

    def test_just_below_mark(self):
        self.assertFalse(is_passing(39))

    def test_top_band(self):
        self.assertEqual(get_grade(95), "Distinction")

    def test_first_band(self):
        self.assertEqual(get_grade(60), "First")

    def test_second_band(self):
        self.assertEqual(get_grade(45), "Second")

    def test_below_all_bands(self):
        self.assertEqual(get_grade(30), "Fail")

    # Activity C: Add one new test (Student-designed)[cite: 9, 19]
    def test_perfect_score_upper_bounds(self):
        self.assertEqual(get_grade(100), "Distinction")

if __name__ == '__main__':
    unittest.main(argv=[""], exit=False)

```